# Linux Service and Process Supervisor

**Team ID: 22 | Section: S9 | Members: G. Jothi Ganesh – 2520030252, V. Nitish Chowdhary – 2520030179, P. Santhosh – 2520030330 | Guide: Dr. Y. Harika Devi**

**Operating Systems and Systems Programming (OSSP – 25CS2104E) | KLH Bachupally | 2026–27, Term-I**

## Abstract

- In multitasking Linux systems, background daemons and services can terminate unexpectedly, hang, or leak memory when managed manually. Designed for Linux administrators and developers, **Linux Service and Process Supervisor** automates service lifecycle control in **C** using `fork()`, `execvp()`, `waitpid()`, POSIX signals, pipes/FIFOs, `shm_open()`, `pthread` pools, and `/proc` telemetry.

**Key Outcome:** Ensures continuous service availability via sub-200ms crash detection, policy-driven auto-restart, and unified CLI, C Socket (`:8080`), and React/WebSocket (`:3000`) supervision.

## Problem Statement & Objectives

- **Problem:** Unsupervised Linux background processes may crash abruptly (`SIGKILL` / `SIGSEGV`), become unresponsive, or create zombies without automated recovery or centralized visibility.

**Objective 1:** Create, track, and synchronize child services in C using `fork()`, `execvp()`, `getpid()` / `setpgid()`, and non-blocking `waitpid(WNOHANG)`.

**Objective 2:** Implement lifecycle actions (`Start`, `Stop`, `Pause`, `Resume`, `Kill`) via POSIX signals (`SIGTERM`, `SIGSTOP`, `SIGCONT`, `SIGCHLD`) with auto-restart.

**Objective 3:** Expose live `/proc/[pid]` memory/FD metrics and IPC streams (`pipe()`, `mkfifo()`, `shm_open()`) across a 40-option CLI and live Web Console.

**Scope:** Single-host Ubuntu/WSL user-space supervision (up to 64 concurrent services across CO-1 to CO-6); excludes kernel-module modification.

## Existing vs Proposed System

| Aspect | Basic / Existing Approach | Proposed Project |
|---|---|---|
| **Process handling** | Manual CLI commands (`ps`, `top`, `kill`) needing human action; heavy init suites add overhead. | Lightweight C engine (`fork()` + `execvp()`) managing 64 services (`services.conf`) with 8-state FSM & Priority/RR/FCFS queues. |
| **Communication** | Isolated processes with no unified stdout capture, external command pipe, or shared live state. | Per-child `pipe()` stdout capture, `/tmp/supervisor_fifo` command IPC, POSIX `shm_open()`, & WebSocket API. |
| **Resource handling** | Manual ad-hoc inspection of RAM and FDs; high risk of zombie accumulation and leaked descriptors. | 4-worker `pthread` pool + `rwlock` continuously parses `/proc/[pid]/status`, `maps`, & `fd` (`VmRSS`, faults). |
| **Error handling** | Crashed services remain offline silently until an operator manually notices and restarts them. | `SIGCHLD` + 200ms watchdog (`waitpid(WNOHANG)`) distinguishes intentional stops vs crashes & auto-restarts failed PIDs. |
| **Key gap** | Disconnected terminal utilities lacking automated fault recovery or real-time browser control. | Unifies low-level C POSIX supervision, IPC, and crash recovery with a 40-cmd CLI & real-time Web GUI. |

## Feasibility Study

- **Technical:** Implemented in GNU C11 (12 modular `.c` / `.h` files compiled with `gcc -lpthread -lrt`) using native Linux POSIX syscalls and `/proc` VFS, paired with Node.js/Express and React 18 on Ubuntu / WSL2.

**Economic:** 100% open-source toolchain (GCC, Make, Linux, Node.js, Git); zero licensing or cloud costs and ultra-low footprint (<5 MB RSS for 4 worker daemons).

**Operational:** Declarative `services.conf` setup; standard user permissions safely control child process groups via terminal CLI or browser UI (`:8080` / `:3000`).

**Schedule:** Completed in 10 weeks within Term-I: W1–2 Design & Config, W3–5 C Core & Signals, W6–8 IPC/Pthreads & Web UI, W9–10 Testing & Docs.

## Tools & Technologies

- **Programming:** C (POSIX Systems Programming — 12 modules), Shell scripting, JavaScript (Node.js/Express & React 18)

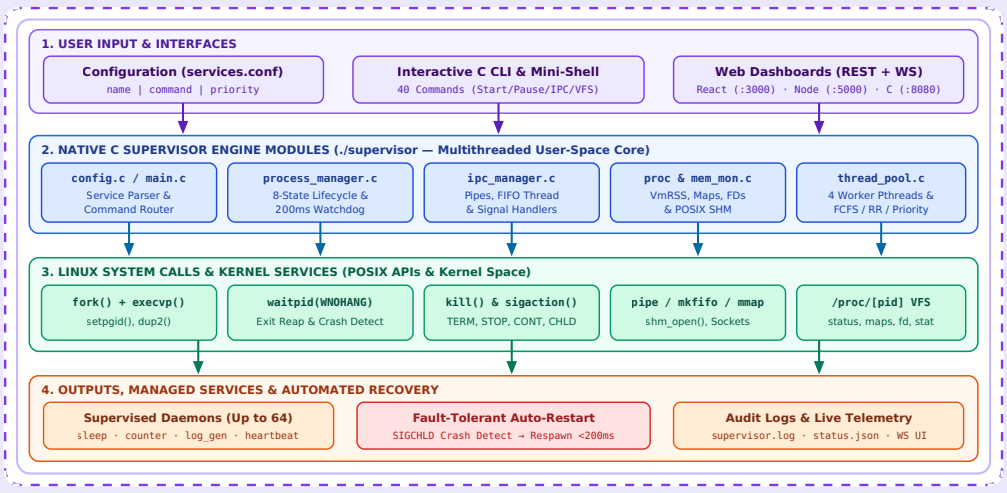
**Platform:** Linux / Ubuntu 22.04 LTS (Native & Windows WSL2)

**OS Concepts:** Process Lifecycle & 8-State FSM, Scheduling (FCFS/RR/Priority), Signals, IPC (Pipes, FIFO, Shared Memory), `/proc` VFS & FDs, Pthreads/Mutex/Semaphores

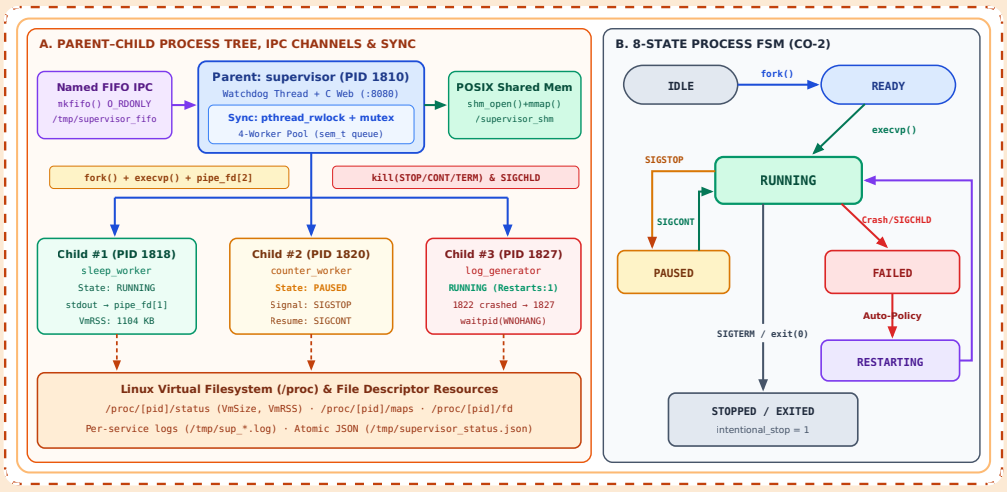
**System Calls / APIs:** `fork()`, `execvp()`, `waitpid()`, `kill()`, `sigaction()`, `pipe()`, `dup2()`, `mkfifo()`, `shm_open()`, `mmap()`, `stat()`, `pthread_create()`, `sem_wait()`

**Tools:** GCC, GNU Make, GDB, POSIX Threads/RT, WebSockets, Git & GitHub, VS Code

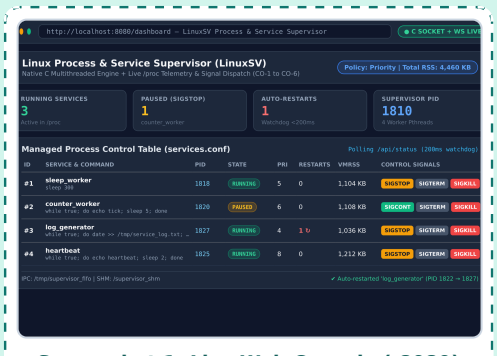
## System Workflow / Architecture



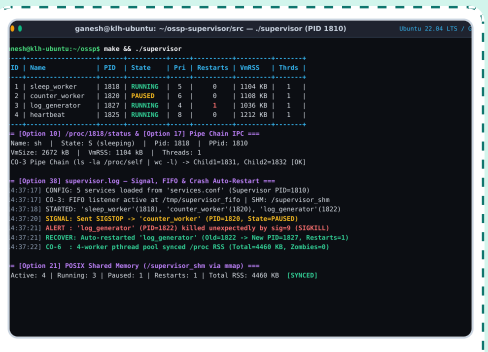

## Process, IPC & Resource Design




## Implementation Screenshots




**Screenshot 1: Live Web Console (:8080) Service States, RSS & Signal Dispatch**




**Screenshot 2: C Supervisor CLI Output /proc Status, Pipe IPC & Auto-Restart**

## Results & Testing

- **Sample Metrics:** Supervised 5 services (`PID 1810`); `fork()+execvp()` latency `<2 ms`; live `/proc` RSS tracked at `4,460 KB`.
- **Signal & Crash Tests:** Verified `SIGSTOP` (`PAUSED`), `SIGCONT`, & `SIGTERM`. Forced `kill -9 1822` auto-restarted `log_generator` as `PID 1827` (`Restarts=1`) in `<200 ms`.
- **Boundary Cases:** Tested invalid PIDs, empty pipes (`EAGAIN`), `MAX_PROCESSES=64`, & clean `SIGINT` exit with `0` zombies.

## Conclusion

- **What Was Implemented:** A modular, multithreaded **Linux Service & Process Supervisor** in C (~2,100 LOC across 12 files) integrated with a live HTTP/WebSocket web dashboard.
- **Objectives & OSSP Alignment:** Fulfils all objectives across **CO-1 to CO-6**—uniting `fork()` / `execvp()` process control, POSIX signals, `pipe` /FIFO/ `shm_open` IPC, `/proc` VFS inspection, and `pthread` synchronization.

## Future Scope

- **Enhancement 1 (OS Resource Quotas):** Integrate Linux **Control Groups (`cgroups v2`)**, `setrlimit()`, and namespaces to enforce per-service CPU/memory caps and backoff restart policies.
- **Enhancement 2 (Observability & Scale):** Upgrade monitoring to event-driven `epoll()` + `pidfd_open()`, add persistent SQLite metrics, TLS role-based auth, and multi-node remote supervision.

**References:** [1] Linux Programmer's Manual (man7.org: `fork(2)`, `execve(2)`, `waitpid(2)`, `sigaction(2)`, `proc(5)`) [2] A. Silberschatz, P. B. Galvin & G. Gagne, Operating System Concepts, 10th Ed. [3] W. R. Stevens & S. A. Rago, Advanced Programming in the UNIX Environment [4] GNU GCC & POSIX.1-2017 Docs

**GitHub:** github.com/logicCrafter320/KLH_CSE_Y25_SEC9_22_Linux-Service-and-Process-Supervisor | **Roles:** Ganesh (Monitoring & `/proc`), Nitish (Signals & Restart), Santhosh (UI, Logs & Testing)

**QR: GitHub**
Scan for Code & Project Docs